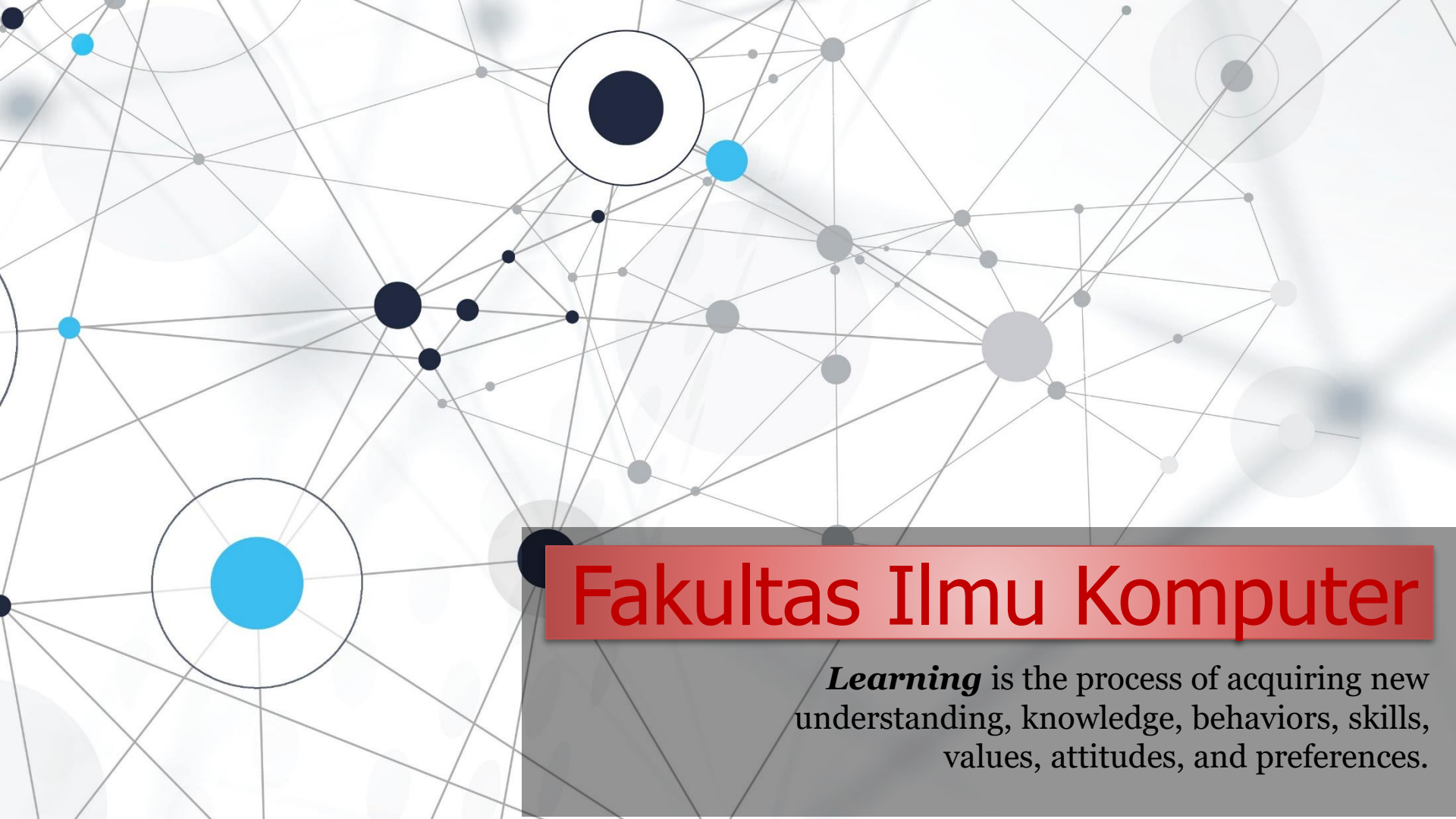


An abstract network diagram with various sized nodes (black, blue, and grey) connected by thin grey lines. Some nodes are highlighted with larger circles. The background is white with faint grey circles.

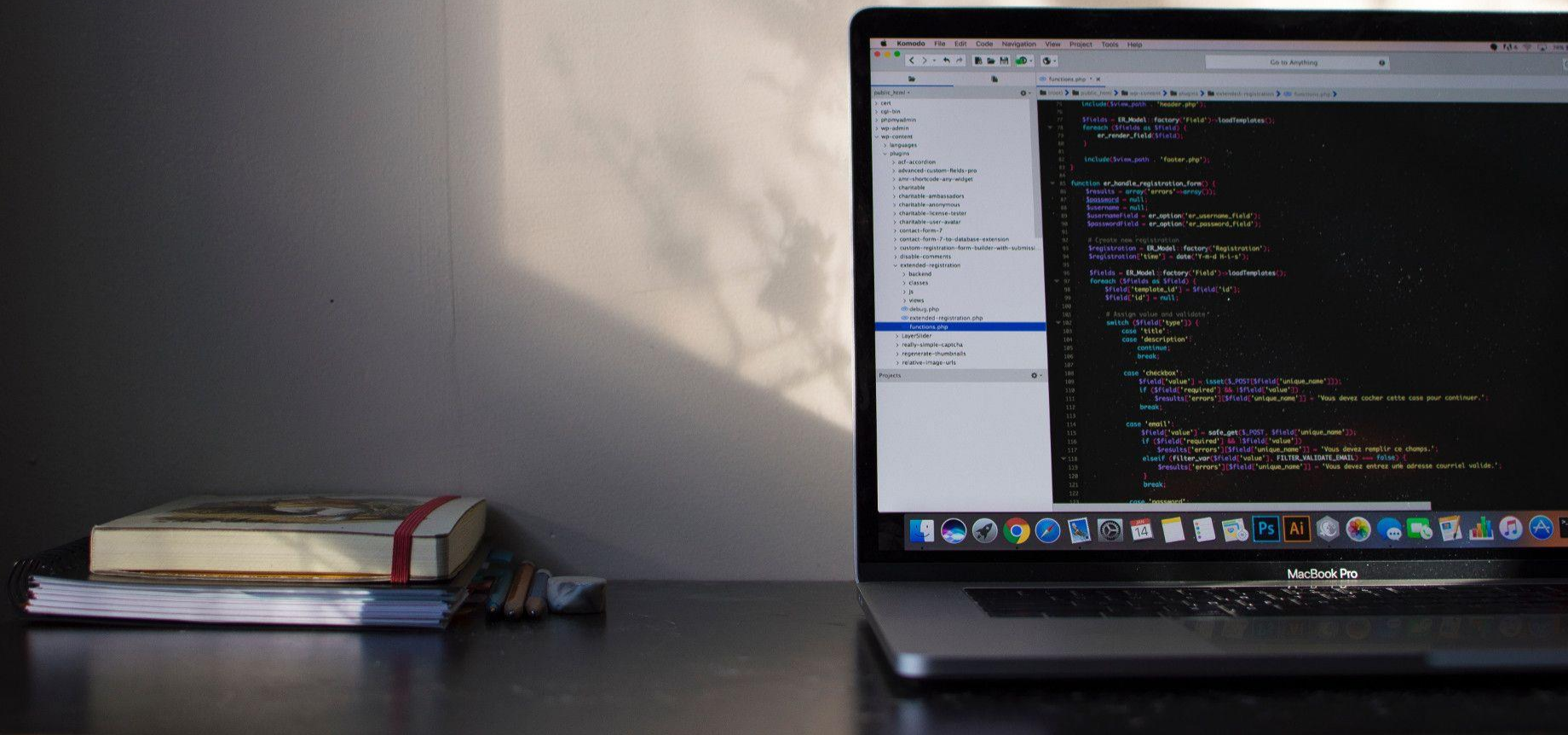
Fakultas Ilmu Komputer

Learning is the process of acquiring new understanding, knowledge, behaviors, skills, values, attitudes, and preferences.

Python OOP - Association



**Welcome to *the* Object Oriented
Programming *course***



Apakah menjadi seorang
developer/*programmer* itu mudah atau sulit?

3:00 AM



Programmer



Menjadi seorang programmer memang memiliki tantangan tersendiri, namun sulit atau tidaknya menjadi seorang programmer sangat tergantung pada individu masing-masing dan juga pada lingkungan di mana mereka bekerja:

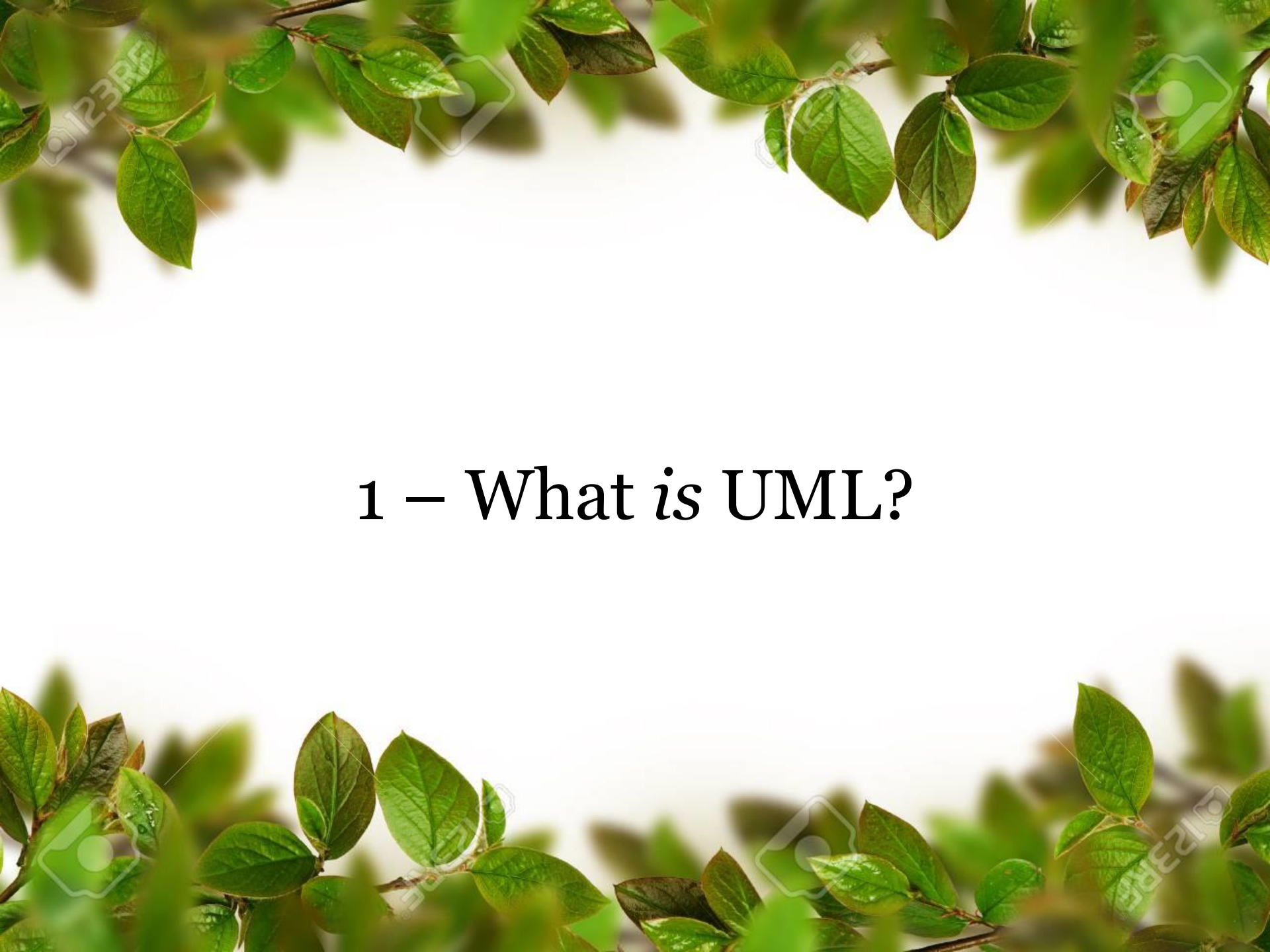
- 1) Memiliki Kurva Belajar yang Tinggi.
- 2) Mencari Solusi untuk Masalah yang Kompleks.
- 3) Kerja Tim yang Intensif.
- 4) Tekanan untuk Memenuhi Deadline.



Programmer

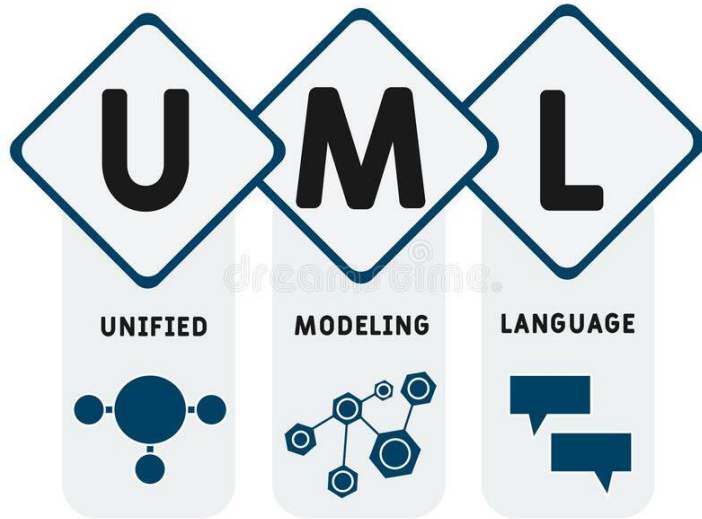


Home

The slide features a white background with green leaves and branches framing the top and bottom edges. The leaves are vibrant green and appear to be from a tree or shrub. The text is centered in the middle of the slide.

1 – What is UML?

Unified Modeling Language (UML)



Unified = Bersatu/Kesatuan
Modeling = Pemodelan/Pola
Language = Bahasa

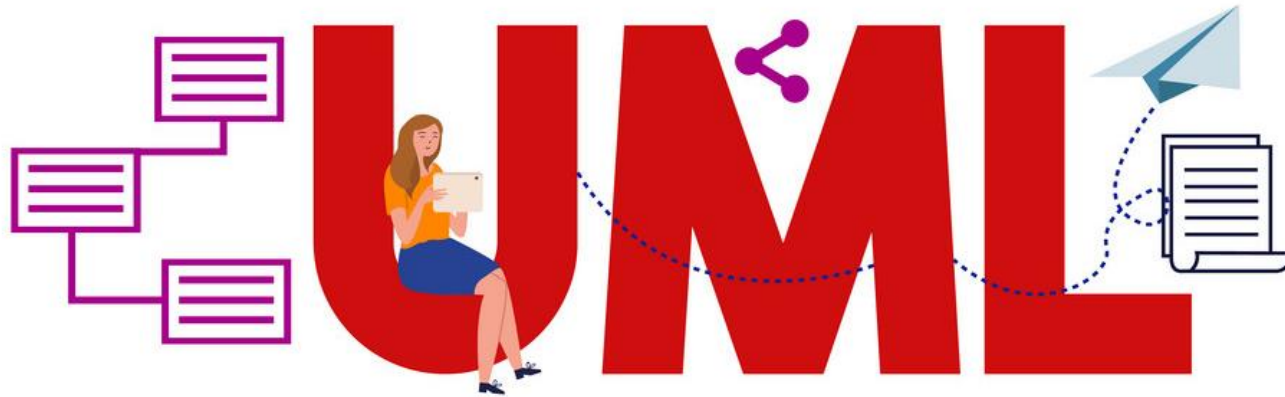
- 1) Unified Modeling Language (UML) adalah sebuah **bahasa pemodelan visual** yang digunakan untuk **mendokumentasikan**, **merancang**, dan **mengkomunikasikan** perangkat lunak yang kompleks.
- 2) UML menggambarkan berbagai aspek dari sistem perangkat lunak seperti **struktur kelas**, **perilaku sistem**, dan **interaksi antara objek**.
- 3) UML diciptakan pada tahun 1997 oleh Grady Booch, Ivar Jacobson, dan James Rumbaugh, yang kemudian bergabung untuk membuat standar notasi umum untuk pemodelan perangkat lunak.

Unified Modeling Language (UML)

- ❖ UML terdiri dari berbagai jenis diagram, seperti **class diagram**, **use case diagram**, **activity diagram**, **sequence diagram**, dan **component diagram**. Setiap jenis diagram UML memiliki tujuan tertentu dalam pemodelan *system/software*.
- ❖ *Why UML is important?* UML sangat penting dalam pengembangan *system/software* karena memungkinkan tim pengembang untuk **berkomunikasi dengan jelas dan efektif tentang rancangan software**, **memahami interaksi antara berbagai bagian sistem**, dan **mengidentifikasi kelemahan dan masalah dalam rancangan sebelum implementasi**.
- ❖ UML juga digunakan sebagai alat untuk dokumentasi dan dokumentasi dapat membantu pengembang *system/software* dalam pemeliharaan dan pengembangan *system/software* di masa depan.

The Unified Modeling Language (UML) is a standard for modeling object-oriented systems. In UML, there are five different types of relationships:

association,
aggregation,
composition,
dependency, and
inheritance.



Mengapa seorang programmer memerlukan pengetahuan tentang UML? Padahal hanya membuat project saja?

Dokumentasi Tekstual vs Dokumentasi Gambar

[1] Textual documentation (dokumentasi tekstual) in system development is a written record that describes the details and details regarding the design, development, and testing of a system or application. This documentation aims to facilitate the understanding of the system being developed by various related parties, such as the development team, project management, and users. Some types of textual documentation that are usually created in system development include:

1) Spesifikasi kebutuhan (*requirement specification*):

Dokumen yang berisi spesifikasi kebutuhan fungsional dan non-fungsional dari sistem yang akan dibangun.

2) Desain sistem (*system design*):

Dokumen yang berisi deskripsi teknis dari struktur sistem, arsitektur, komponen, dan interaksi antara komponen-komponen tersebut.

3) Dokumentasi kode (*code documentation*):

Dokumen yang berisi penjelasan rinci tentang kode program yang telah dibuat, termasuk fungsi, variabel, dan algoritma yang digunakan.

4) Dokumentasi pengujian (*testing documentation*):

Dokumen yang berisi hasil pengujian sistem dan metode yang digunakan untuk melakukan pengujian tersebut.

[2] Visual documentation (UML (Unified Modeling Language)) in system development is a document that uses predefined graphical notations to present and explain the design model of a system or application. This documentation usually uses various types of UML diagrams designed to visually depict various aspects of the system, such as class structure, relationships between classes, use cases, interactions between objects, and so on.

Programmers Need UML

Programmer sangat membutuhkan UML karena UML adalah alat yang sangat berguna dalam membantu *programmer* dalam memahami, merancang, dan mengembangkan sistem/software. Berikut adalah beberapa alasan mengapa *programmer* memerlukan UML:

Simplifies Communication

UML allows programmers to communicate with the development team and other stakeholders about the system design clearly and effectively.

Speeds up Development Process

By using UML, it can help in identifying problems and weaknesses in the design before implementation, minimizing errors, and saving time and cost.

Helps Understand System Structure and Interaction

UML provides various types of diagrams that can help programmers understand the structure and interactions between parts of the system.

System Documentation

UML can be used as a tool for documentation. Programmers can use UML diagrams to create documentation about software systems.

Benefits & Uses of UML

(1) Fleksibel

- ❖ UML is a highly recognized and understood platform for software design. UML is a standard notation that is widely used by developers. When we are just learning about software, we can use UML easily because of its flexible nature, making it an alternative to explain software design models.
- ❖ UML cocok dan sangat dibutuhkan untuk pengembangan software adalah fleksibilitasnya dikarenakan UML dapat menyesuaikan elemen pemodelan dan interaksi dalam diagram UML secara khusus agar sesuai dengan domain atau teknologi yang digunakan.

(2) UML juga digunakan sebagai Alat Perencanaan.

(3) Representasi Visual

Diagram UML adalah representasi visual dari hubungan antara kelas dan entitas dalam program komputer. Kelas adalah objek dalam pemrograman yang mengatur variabel dan fungsi serupa di satu lokasi.

UML sangat penting bagi developer/programmer karena dapat membantu untuk memahami, merancang, dan mengembangkan software/sistem secara lebih efektif dan efisien. Dengan menggunakan UML, developer/programmer dapat membuat sistem yang lebih berkualitas, lebih andal, dan lebih mudah untuk dipelihara.



A decorative border of green leaves and branches at the top of the slide.

2 - Association Concept *in* OOP

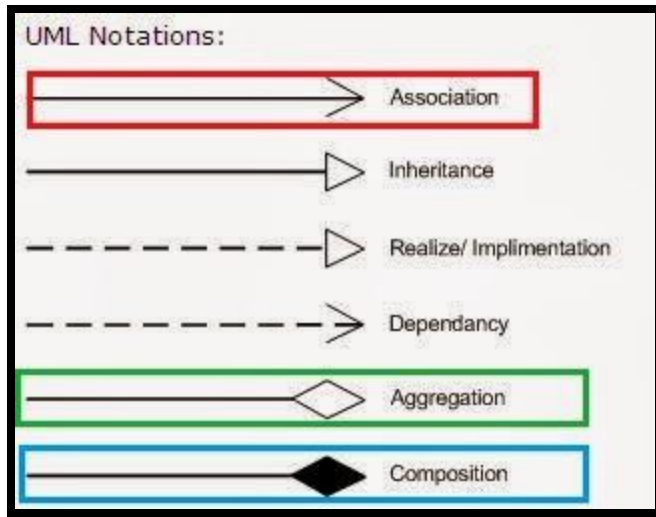
A decorative border of green leaves and branches at the bottom of the slide.

Association in OOP

- ❖ **Association** is a semantically **weak relationship (a semantic dependency)** between otherwise **unrelated objects**. An association is a “using” relationship between two or more objects in which the objects have their own lifetime and there is no owner.
- ❖ In UML an association relationship is represented by a **single arrow**. An association relationship can be represented as **one-to-one**, **one-to-many**, or **many-to-many** (*also known as cardinality*).
- ❖ As an example, imagine the relationship between a doctor and a patient. A doctor can be associated with multiple patients. At the same time, one patient can visit multiple doctors for treatment or consultation. Each of these objects has its own life cycle and there is no “owner” or parent.

An association relationship between two or more objects denotes a path of communication (*also called a link*) between them so that one object can send a message to another.

Association in OOP



Relations

_____◇

_____◆

_____>

----->

Names

Association

Aggregation

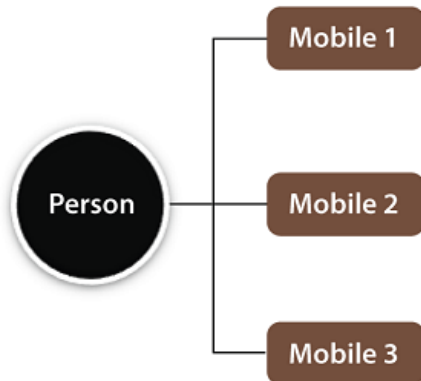
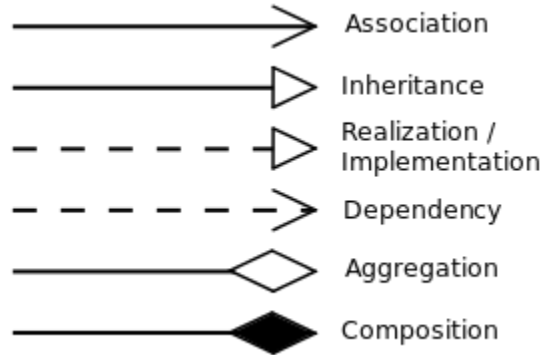
Composition

Inheritance

Realization

Dalam konsep Pemrograman Berorientasi Objek (OOP), asosiasi (*association*) merujuk pada **hubungan antara** dua atau lebih objek yang saling terkait dalam suatu konteks tertentu.

Types of Associations in OOP



- 1) Asosiasi biasa (*ordinary association*): objek memiliki hubungan yang terdefinisi secara jelas dan objek tersebut tidak bergantung satu sama lain.
- 2) Agregasi (*aggregation*): satu objek memiliki beberapa objek terkait, tetapi objek-objek tersebut dapat ada tanpa objek utama.
- 3) Komposisi (*composition*): satu objek terdiri dari beberapa objek terkait, dan objek-objek tersebut hanya dapat ada dengan objek utama.
- 4) Inheritance (*pewarisan*): objek turunan mewarisi sifat dan perilaku dari objek induk.

Penggunaan asosiasi yang tepat dapat membantu mengorganisir kode program dan membuat program lebih mudah dipahami dan dikembangkan.

Association in OOP

Dalam **asosiasi**, dua atau lebih objek saling berinteraksi dan bekerja sama untuk mencapai tujuan tertentu. Hubungan antara objek dapat bersifat satu arah (*unidirectional*) atau saling berpengaruh (*bidirectional*).

Sifat satu arah
(unidirectional)
association pada UML menunjukkan bahwa hubungan antara dua kelas berjalan ke satu arah atau arah tertentu saja. Artinya, kelas yang diberi panah pada asosiasi hanya dapat mengakses kelas lain dan tidak sebaliknya.

Sifat saling berpengaruh
(bidirectional)
association pada UML menunjukkan bahwa hubungan antara dua kelas berjalan ke dua arah atau saling berpengaruh. Artinya, kedua kelas yang terlibat dalam asosiasi dapat mengakses atribut dan metode satu sama lain.

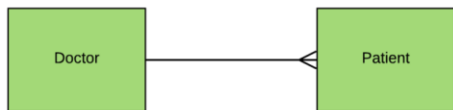
One-to-one Association Class Diagram

One-to-one association adalah jenis asosiasi di antara dua kelas di mana setiap objek pada kelas pertama memiliki satu dan hanya satu objek pada kelas kedua, dan setiap objek pada kelas kedua juga hanya memiliki satu dan hanya satu objek pada kelas pertama. Artinya, setiap objek pada kelas pertama terhubung dengan satu objek pada kelas kedua secara eksklusif dan saling bergantung.

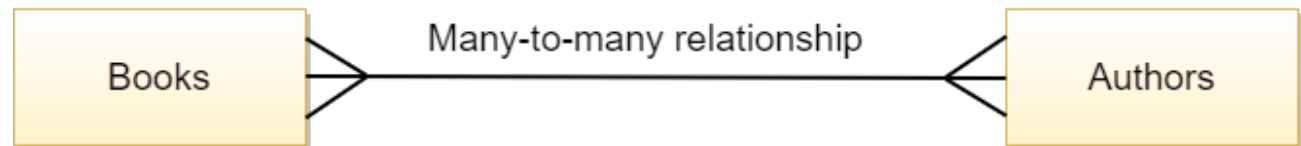
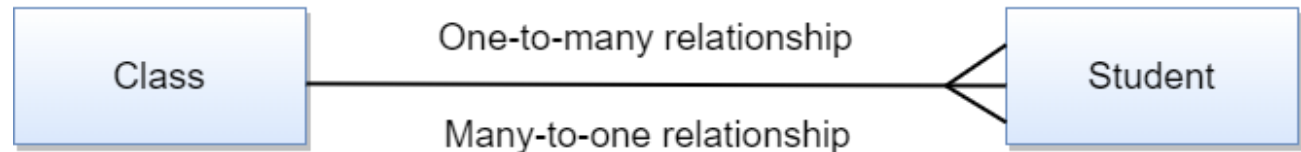
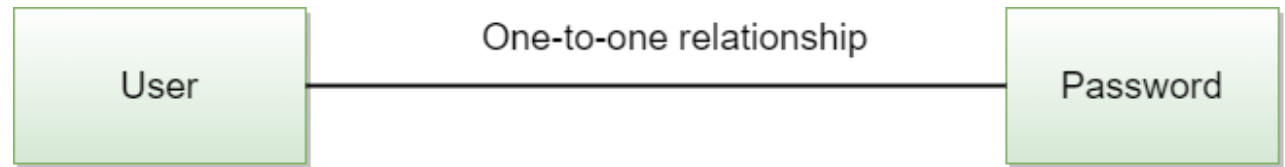
One to One



One to Many



Many to Many



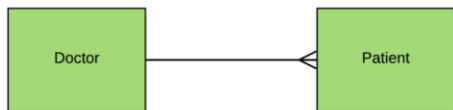
One-to-many Association Class Diagram

One-to-many association adalah jenis asosiasi antara dua kelas di mana setiap objek pada kelas pertama memiliki banyak objek yang terkait pada kelas kedua, tetapi setiap objek pada kelas kedua hanya terhubung dengan satu objek pada kelas pertama. Artinya, setiap objek pada kelas pertama memiliki banyak objek pada kelas kedua yang terkait, tetapi setiap objek pada kelas kedua hanya terkait dengan satu objek pada kelas pertama.

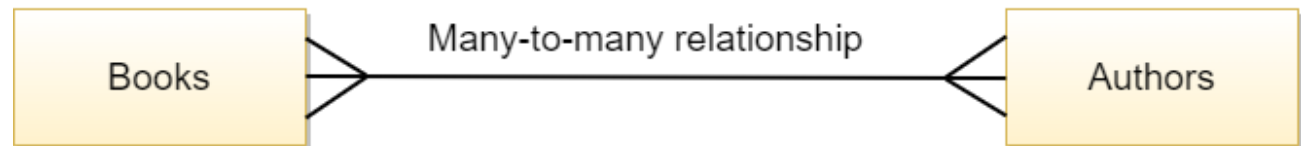
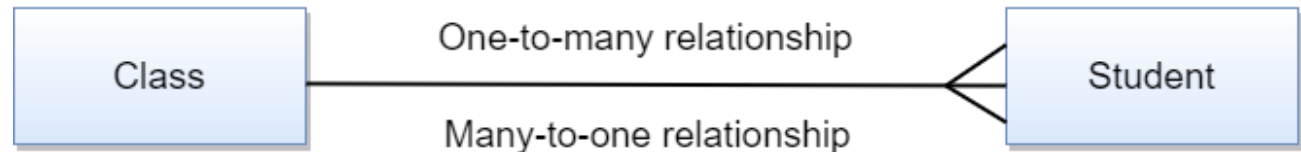
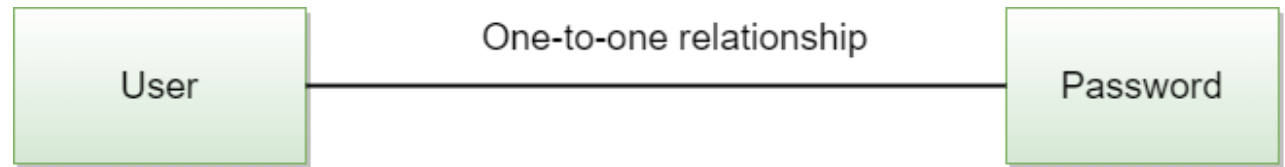
One to One



One to Many

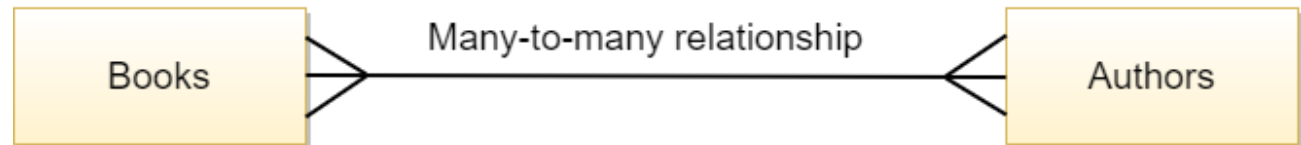
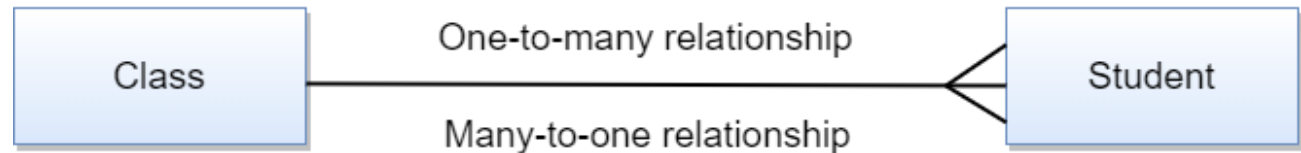
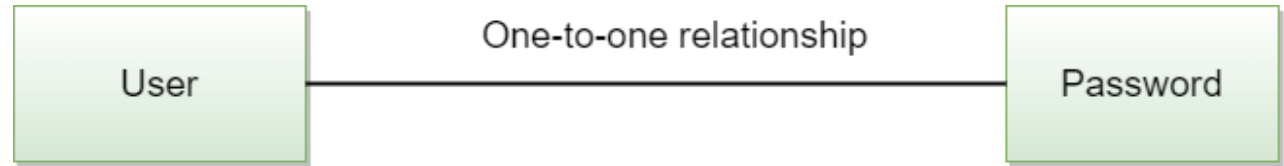
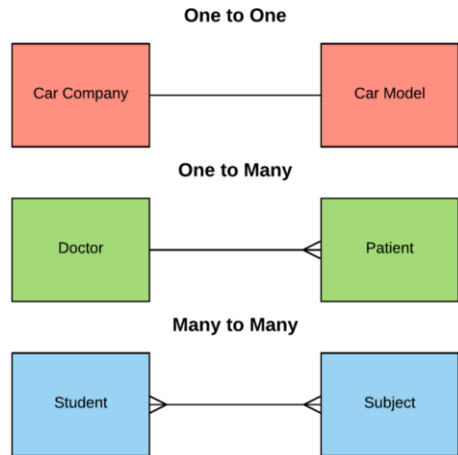


Many to Many



Many-to-many Association Class Diagram

Many-to-many association adalah jenis asosiasi antara dua kelas di mana banyak objek pada kelas pertama terkait dengan banyak objek pada kelas kedua. Artinya, banyak objek pada kelas pertama dapat terhubung dengan banyak objek pada kelas kedua dan sebaliknya.





Exercise *for* Students

Class Exercise #1

```
1  # create class #1
2  class Penulis:
3      def __init__(self, nama, alamat):
4          self.nama = nama
5          self.alamat = alamat
6
7      def tulis_buku(self, judul):
8          print(f"{self.nama} menulis buku dengan judul {judul}")
9
10 # creae class #2
11 class Buku:
12     def __init__(self, judul, penulis):
13         self.judul = judul
14         self.penulis = penulis
15
16     def cetak_info(self):
17         print(f"Buku '{self.judul}' ditulis oleh {self.penulis.nama} dari {self.penulis.alamat}")
18
19
20 # create objects dari class Penulis dan class Buku
21 penulis1 = Penulis("Semmy Wellem Taju", "Manado")
22 buku1 = Buku("Java Programming", penulis1)
23
24 # info buku
25 buku1.cetak_info()
26
27 # info penulis
28 penulis1.tulis_buku("Java Programming")
```

Setiap penulis dapat menulis satu buku, sehingga ada **hubungan one-to-one** antara penulis dan buku.

Perhatikan Exercise #1,
Dikarenakan setiap **penulis** dapat
menulis satu **buku**, sehingga disebut **one-
to-one association** antara penulis dan
buku.

Bagaimana cara untuk mengubah ke
dalam bentuk **one-to-many association**
(satu penulis dapat memiliki lebih dari
satu buku yang terkait dengannya)??
Apakah bisa???

```

1 # create class #1
2 class Penulis:
3     def __init__(self, nama):
4         self.nama = nama
5         self.daftar_buku = [] # empty array
6
7     def tulis_buku(self, judul):
8         buku = Buku(judul, self.nama) # create object buku
9         self.daftar_buku.append(buku)
10
11 # create class #2
12 class Buku:
13     def __init__(self, judul, penulis):
14         self.judul = judul
15         self.penulis = penulis
16
17 # object dari class Penulis
18 penulis1 = Penulis("Semmy Taju")
19
20 # daftar buku-buku
21 penulis1.tulis_buku("Java Programming")
22 penulis1.tulis_buku("Data Structures in Python")
23 penulis1.tulis_buku("Machine Learning Algoritms")
24
25 # Mengakses daftar buku dan informasi penulis
26 for buku in penulis1.daftar_buku:
27     print("Judul Buku:", buku.judul)
28     print("Penulis Buku:", buku.penulis)
29     print()
30

```

Sekarang ini,
satu penulis
dapat memiliki
lebih dari satu
buku yang
terkait
dengannya.

one-to-many association

```

Judul Buku: Java Programming
Penulis Buku: Semmy Taju

```

```

Judul Buku: Data Structures in Python
Penulis Buku: Semmy Taju

```

```

Judul Buku: Machine Learning Algoritms
Penulis Buku: Semmy Taju

```

Buatlah **many-to-many association**
dengan menggunakan code di slide
sebelumnya.

END PRESENTATION

Thank you for your attention

Instructor: S – W – T

THANK YOU

